# Examen virtual

# Pilas, colas y listas

**Emmanuel Gaviria Acevedo**
**Miguel Angel Tobon Zapata**

**Tecnología en Desarrollo de Software**

**Instituto Pascual Bravo**

**Medellín, 2026**

# PUNTO 1

**Ejercicio 1: Pila y cola**

**Estado inicial:** P (tope→base) = 9, 4 · Q (frente→final) = 6, 2
**Estado objetivo:** P (tope→base) = 2, 4 · Q (frente→final) = 6, 9

**a. Secuencia propuesta**

M, R, N, R (4 movimientos, dentro del límite de 5)

**b. Estados después de cada movimiento**

**ESTADO INICIAL**

**P**: tope → [9] [4] ← base **Q:** frente → [6] [2] ← final

**1) M (pop de P = 9, enqueue en Q) valor trasladado: 9**

**P:** tope → [4] ← base **Q:** frente → [6] [2] [9] ← final

**2) R (dequeue de Q = 6, enqueue en Q) valor trasladado: 6**

**P: tope → [4] ← base Q: frente → [2] [9] [6] ← final**

**3) N (dequeue de Q = 2, push en P) valor trasladado: 2**

**P**: tope → [2] [4] ← base **Q**: frente → [9] [6] ← final

**4) R (dequeue de Q = 9, enqueue en Q) valor trasladado: 9**

**P**: tope → [2] [4] ← base **Q:** frente → [6] [9] ← final ✓ OBJETIVO

**c. Pseudocódigo del movimiento N**

```
procedimiento MovimientoN(P, Q)
  si Q.estaVacia() entonces
```

```
    error "No se puede desencolar de una cola vacía"
  fin si
  valor ← Q.dequeue()     // retira el elemento del frente de Q
  P.push(valor)           // lo coloca en el tope de P
fin procedimiento
```

(Para M sería valor ← P.pop() seguido de Q.enqueue(valor); para R, valor ← Q.dequeue() seguido de Q.enqueue(valor).)

d. Justificación

Se necesitan dos cambios: que el 9 pase de P al final de Q, y que el 2 pase de Q al tope de P. El movimiento M saca el 9 (que está en el tope, así que es accesible) y lo deja en Q. Luego el 2 no está en el frente de Q porque el 6 está delante, así que R rota el 6 al final, dejando el 2 en el frente. N pasa el 2 al tope de P, encima del 4, y P queda exactamente 2, 4. Por último, Q queda 9, 6, pero el objetivo es 6, 9; un R más rota el 9 al final y Q queda 6, 9. En ningún paso se crearon, duplicaron ni descartaron datos, nunca se retiró de una estructura vacía y solo se accedió a tope y frente. Usamos 4 movimientos, menos que el máximo de 5.

Revisión de la propuesta del otro equipo: M, N, R

**INICIAL P: [9] [4] Q: [6] [2]**

**M (9) P: [4] Q: [6] [2] [9]**

**N (6) P: [6] [4] Q: [2] [9]**

**R (2) P: [6] [4] Q: [9] [2]**

Al ejecutar N se retira el 6, porque es el que está en el frente de Q en ese momento, no el 2. Por eso la propuesta no logra el objetivo: termina con P = 6, 4 (debía ser 2, 4) y Q = 9, 2 (debía ser 6, 9). El error está en hacer N antes de rotar la cola; faltaba un R entre M y N para que el 2 llegara al frente, y otro R al final para ordenar Q.

# PUNTO 2

**Diagnóstico y reparación de enlaces**

**2.1 Lista simple**

**Estado defectuoso:**

cabeza

↓

[A] —sig—> [B] —sig—> [C]

↑ |

└———sig———┘ ✗ C.siguiente = B

**Referencia incorrecta:** C.siguiente = B. En una lista lineal el último nodo debe apuntar a nulo. Esta referencia crea un ciclo B ↔ C.

**Recorrido defectuoso (máx. 5 visitas):** A → B → C → B → C. El recorrido nunca llega a nulo.

**Reparación:**

[A] —sig—> [B] —sig—> [C] —sig—> nulo (1) C.siguiente = nulo

**Por qué funciona:** C pasa a ser el último nodo y el recorrido termina. El recorrido queda A → B → C → nulo.

---

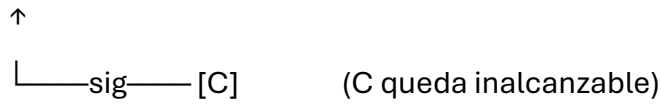
**2.2 Lista circular simple**

**Estado defectuoso:**

cabeza

↓

[A] —sig—> [B] —sig—> nulo ✗ B.siguiente = nulo

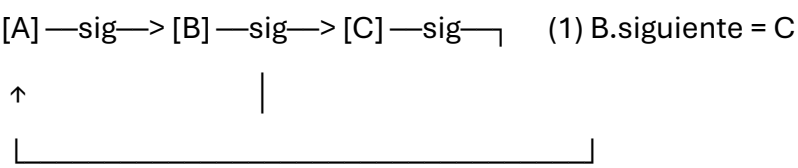



(C queda inalcanzable)

**Referencia incorrecta:** B.siguiente = nulo. Debería apuntar a C. Por eso C no se puede alcanzar y el círculo no se cierra.

**Recorrido defectuoso:** A → B, y se detiene al llegar a nulo (2 visitas).

**Reparación:**




(1) B.siguiente = C

**Por qué funciona:** B vuelve a enlazar con C, y C ya apuntaba a A, así que el círculo queda cerrado. El recorrido queda A → B → C → A → B.

---

**2.3 Lista doble lineal**

**Estado defectuoso:**

siguiente: [A] —> [B] —> [C] —> nulo

anterior: nulo <— [A] [B] —> [C] ✗ B.anterior = C

[B] <— [C]

**Referencia incorrecta:** B.anterior = C. Debería ser A, porque A es el nodo que precede a B. Así se rompe la simetría A.siguiente = B ⇔ B.anterior = A.

**Recorrido desde cola con anterior:** C → B → C → B → C. Entra en un ciclo y nunca llega a A.

**Reparación:**

anterior: nulo <— [A] <— [B] <— [C] (1) B.anterior = A

**Por qué funciona:** cada anterior es ahora el inverso de un siguiente. El recorrido queda C → B → A → nulo.

---

**2.4 Lista doble circular**

**Valores correctos esperados:**

- siguiente: A→B, B→C, C→A
- anterior: A→C, B→A, C→B

**Estado defectuoso:**

siguiente: [A] —> [B] —> [C] —> (A) ✓ correcto

anterior: [A] —> [B] ✗ A.anterior = B

[B] —> [A] ✓

[C] —> [B] ✓

**Referencia incorrecta:** A.anterior = B. Debería ser C. Como C.siguiente = A, el anterior de A tiene que ser C.

**Recorrido desde cabeza con anterior:** A → B → A → B → A. El recorrido queda atrapado entre A y B, y C nunca se visita.

**Reparación:**

anterior: [A] —> [C] (1) A.anterior = C

A <— B <— C <— A (circular)

**Por qué funciona:** se cierra el círculo hacia atrás, de modo que anterior es el inverso exacto de siguiente. El recorrido queda A → C → B → A → C.

---

**Resumen**

| Caso | Referencia incorrecta | Recorrido defectuoso | Corrección |
|---|---|---|---|
| 2.1 | C.siguiente = B | A, B, C, B, C | C.siguiente = nulo |
| 2.2 | B.siguiente = nulo | A, B (nulo) | B.siguiente = C |
| 2.3 | B.anterior = C | C, B, C, B, C | B.anterior = A |
| 2.4 | A.anterior = B | A, B, A, B, A | A.anterior = C |

**Avance 2: commit y push**

Guarden lo anterior en un archivo, por ejemplo avance2/enlaces_reparados.md, y luego ejecuten:

bash

git add avance2/enlaces_reparados.md

git commit -m "Avance 2: diagnóstico y reparación de enlaces en las cuatro listas"

git push

# PUNTO 3

Elección y adaptación de estructuras

1. Solicitudes → Cola (FIFO)

Justificación: las solicitudes se atienden en orden de llegada, así que la primera en entrar es la primera en salir. Una cola encolada por final y desencolada por frente cumple exactamente esa regla, y ambas operaciones son O(1) si se implementa con una lista enlazada que guarde referencias a frente y final.

Ejemplo (llegaron S1, luego S2, luego S3):

frente final

↓ ↓

[S1] —sig—> [S2] —sig—> [S3] —sig—> nulo

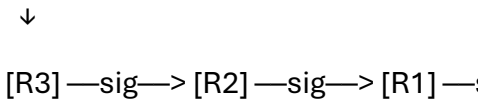
desencolar() → se atiende S1 primero

2. Deshacer → Pila (LIFO)

Justificación: se revierte primero la reasignación más reciente, así que la última en entrar es la primera en salir. Una pila con apilar y desapilar sobre la cima cumple esa regla en O(1).

Ejemplo (se hicieron R1, luego R2, luego R3):

cima

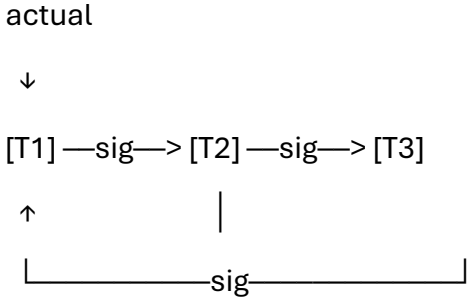



desapilar() → se deshace R3 primero

3. Técnicos

Antes del cambio → Lista circular simple

Justificación: los turnos se repiten y después del último técnico se vuelve al primero. Como solo se avanza, basta un enlace siguiente, y el último nodo apunta al primero. Una referencia actual indica a quién le toca, y avanzar es actual = actual.siguiente.




Después del cambio → Lista doble circular

Justificación: ahora también hay que retroceder, incluso desde el primero al último. En la lista simple eso exigiría recorrer toda la lista (O(n)). Al agregar un enlace anterior en cada nodo, retroceder es O(1) y el cierre circular se mantiene en ambos sentidos.

Versión final con todos sus enlaces:

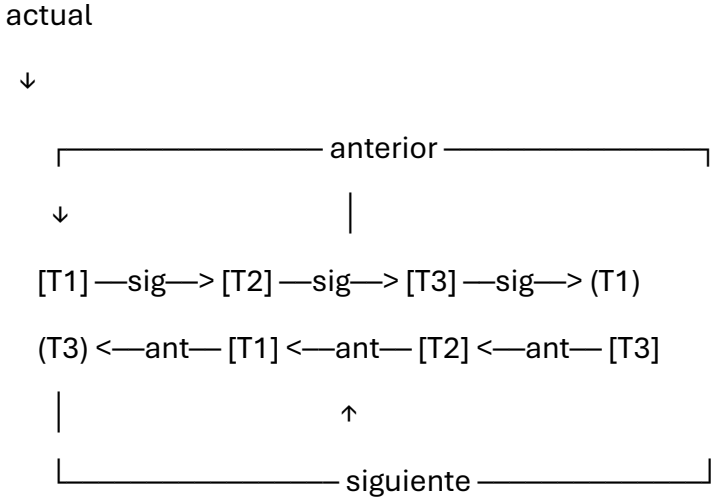


Tabla de enlaces:

| Nodo | siguiente | anterior |
|---|---|---|
| T1 | T2 | T3 |
| T2 | T3 | T1 |
| T3 | T1 | T2 |

Explicación del cambio y la información adicional

Cada nodo ahora guarda tres cosas:

- el dato del técnico (identificador y nombre),
- siguiente, que ya existía,
- anterior, que es la referencia adicional.

Hay que mantener siempre la simetría X.siguiente = Y ⇔ Y.anterior = X. El cierre se da con T3.siguiente = T1 y T1.anterior = T3, y este último enlace es el que permite pasar del primero al último en un solo paso. Avanzar sigue siendo actual = actual.siguiente, y retroceder es actual = actual.anterior.

La contrapartida es que cada nodo ocupa una referencia más de memoria. Además, al insertar o eliminar un técnico hay que actualizar cuatro enlaces en lugar de dos.

# PUNTO 4

1. Nodo que se quiere retirar: B

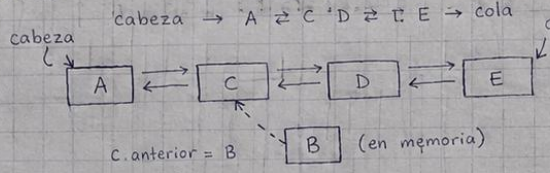
Actualización ejecutada: (casos[integrante].modifica)

A.siguiente = C

La cadena hacia adelante ya no incluye el nodo retirado, pero el enlace anterior del nodo siguiente sigue apuntando al nodo retirado.

Dibujo inmediatamente después del error

cabeza → A ⇄ C ⇄ D ⇄ E → cola




2. Primeras tres visitas al recorrer hacia atrás

Se comienza en el nodo que estaba después del nodo que se quiere retirar.

(visita 1) C → (visita 2) B → (visita 3) A

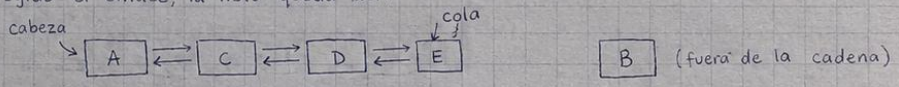
El recorrido encuentra el nodo que se pretendía retirar porque su enlace anterior no se actualizó.

3. Reparación

La referencia que falta actualizar es:

C.anterior = A

Una vez corregido el enlace, la lista queda así:




El nodo retirado queda fuera de la cadena activa.

---

1. Nodo que se quiere retirar: C

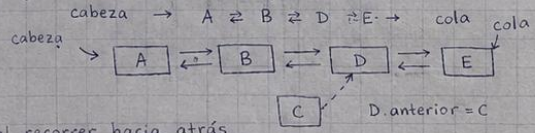
Actualización ejecutada: (casos[integrante].modifica)

B.siguiente = D

La cadena hacia adelante ya no incluye el nodo retirado, pero el enlace anterior del nodo siguiente sigue apuntando al nodo retirado.

Dibujo inmediatamente después del error

cabeza → A ⇄ B ⇄ D ⇄ E → cola




2. Primeras tres visitas al recorrer hacia atrás

Se comienza en el nodo que estaba después del nodo que se quiere retirar.

(visita 1) D → (visita 2) C → (visita 3) B

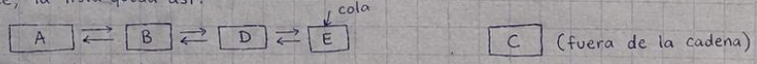
El recorrido encuentra el nodo que se pretendía retirar porque su enlace anterior no se actualizó.

3. Reparación

La referencia que falta actualizar es:

D.anterior = B

Una vez corregido el enlace, la lista queda así:




El nodo retirado queda fuera de la cadena activa.

Punto A: Emmanuel Gaviria- Punto B: Miguel Tobon